

On-Chain Diploma Verification on Ethereum: Cost Trade-Offs Between Per-Record Storage and Merkle-Batch Architectures

Nazar Shcherbyna

Faculty of Electrical Engineering

Warsaw University of Technology, Warsaw, Poland

nazar.shcherbyna.stud@pw.edu.pl

Abstract—This paper presents UniVerify, a prototype system for issuing, verifying, and revoking academic diplomas on Ethereum. The study compares two on-chain representations: a *packed per-record benchmark baseline*, where each diploma occupies one packed storage slot, and *Merkle batch issuance*, where an entire graduation cohort is represented by a single Merkle root.

To separate architectural and implementation effects, four contract variants are benchmarked: a packed per-record model (**Packed**), a custom Solidity Merkle implementation (**Baseline**), the OpenZeppelin MerkleProof library (**OpenZeppelin**), and an assembly-optimized Merkle variant (**Optimized**). Measurements cover proof depths $d \in \{0, 1, 4, 8, 10, 12\}$.

Merkle batching reduces per-diploma issuance cost by 98.9% at batch size $N = 100$, while shifting part of the cost to proof-based verification. Compared with **Baseline**, **Optimized** lowers verification gas by 6.6%, and by 3.7% relative to **OpenZeppelin**. A closed-form gas-savings model is derived and validated at six proof depths, with residuals below 0.2%. Under an illustrative mainnet scenario (40 gwei, 1 ETH = \$2,500), issuing 1,000 diplomas costs about \$4,820 with per-record storage versus about \$5.1 with Merkle batching. Overall, the results indicate that for institutional diploma issuance, batching has a much larger impact on practical cost than low-level micro-optimization.

Index Terms—Ethereum, Solidity, gas optimization, Merkle tree, smart contracts, diploma verification

I. INTRODUCTION

Academic credential verification remains largely institution-centric. Cross-border or post-graduation checks often require manual contact with the issuing university, which makes the process slow and difficult to automate. At the same time, forged diplomas remain a persistent problem [1]. Blockchain-based registries offer an alternative: once a credential commitment is published on-chain, any verifier can validate authenticity independently, without relying on a single institutional database.

The main challenge is economic. On Ethereum, persistent storage writes are expensive. If each diploma is stored individually, the university pays for one state update per record. For institutions issuing hundreds or thousands of diplomas per cycle, this quickly becomes the dominant cost. The key question is therefore not whether blockchain can be used, but *how credentials should be represented on-chain so that issuance remains practical*.

This paper studies that question in the context of *UniVerify*, a prototype diploma-verification system. Two architectures are compared:

- 1) **Per-record storage**, where each diploma is stored in its own mapping slot and verified in constant time.
- 2) **Merkle batch issuance**, where one Merkle root represents an entire batch and each diploma is verified using an off-chain proof.

The contribution is practical rather than conceptual. Rather than proposing a new credential format, this work quantifies the gas implications of deployable Solidity designs and asks: *when does batching make on-chain diploma registries economically viable?*

A. UniVerify System

UniVerify is a prototype system for issuing, verifying, and revoking academic diplomas on Ethereum. It consists of:

- a **Solidity smart contract** for on-chain storage, verification, and revocation;
- an **off-chain TypeScript SDK** for hashing, Merkle tree construction, and proof generation;
- a **web application** for issuers, verifiers, and administrators;
- a **REST API** for integration with university information systems.

Universities act as *issuers* who publish diploma commitments on-chain. A verifier later checks a diploma using the document hash and, in the Merkle variant, a proof of inclusion in the published batch root. In the current UniVerify contract, the final `Valid` decision also depends on whether the issuer remains linked to an active university at the time of verification.

B. Contributions

- 1) A quantitative comparison between a packed per-record benchmark baseline and a Merkle-batch UniVerify registry across issuance, verification, revocation, and deployment cost.
- 2) A four-way benchmark of **Packed**, **Baseline**, **OpenZeppelin** [9], and **Optimized**, separating architectural, library-level, and low-level EVM effects.
- 3) A closed-form gas-savings model for the optimized Merkle path, validated experimentally at six proof depths with residuals below 0.2%.
- 4) An economic interpretation showing how gas savings translate into institutional issuance cost.

II. RELATED WORK

Grech and Camilleri [2] identified credential verification as one of the main educational applications of blockchain. Lizcano et al. [4] proposed a decentralized framework for academic credentials based on IPFS and Ethereum, but without low-level cost analysis. Alketbi et al. [3] discussed broader blockchain use in public-sector services, including education records.

Merkle trees [5] are the standard mechanism for compact set-membership proofs. OpenZeppelin’s `MerkleProof` [9] serves as the de facto production Solidity library. Alternative authenticated data structures, including sparse Merkle trees, Verkle trees [10], and RSA accumulators, remain interesting but less practical for ordinary EVM contracts.

Ethereum execution costs are determined by the EVM gas schedule [8]. In particular, EIP-2929 increased the cost of cold state accesses, directly affecting storage-based lookups [6]. Most prior work on blockchain credentials emphasizes system architecture, privacy, governance, or interoperability. By contrast, this paper studies a narrower but practically important question: how specific Solidity design choices change issuance and verification cost in a deployable credential-registry prototype.

III. SYSTEM DESIGN AND METHODOLOGY

A. Two Verification Architectures

The two approaches are illustrated in Fig. 1.

Architecture A — Per-Record Storage. For benchmarking, each diploma is stored in a mapping (`bytes32` $\Rightarrow$ `uint256`). The issuer address (160 bits) and a revocation flag (1 bit) are packed into one 256-bit slot. Issuance requires one `SSTORE` per diploma. Verification is a single `SLOAD` plus bit masking, so read cost is constant. This contract is intentionally minimal and serves as a gas baseline rather than a feature-equivalent version of the full UniVerify registry.

Architecture B — Merkle Batch. All diplomas in a cohort are arranged into a Merkle tree off-chain. Only the 32-byte batch root is stored on-chain via `issueBatchRoot`, so one state write covers the entire batch. Verification requires the caller to provide $d = \lceil \log_2 N \rceil$ sibling hashes, shifting cost from storage to proof processing. Each leaf is domain-separated: `keccak256(contract||chainId||issuer||batchId||docHash)`, which prevents cross-contract and cross-chain replay.

B. Contract Variants

To distinguish architectural effects from implementation effects, four contract variants were implemented (Table I).

C. Optimization Techniques

The **Optimized** variant targets two internal functions used during proof verification: `_processProof()` and `_merkleLeaf()`.

T1 — Unchecked loop arithmetic. Wrapping the loop increment in `unchecked{++i;}` removes overflow-check overhead.

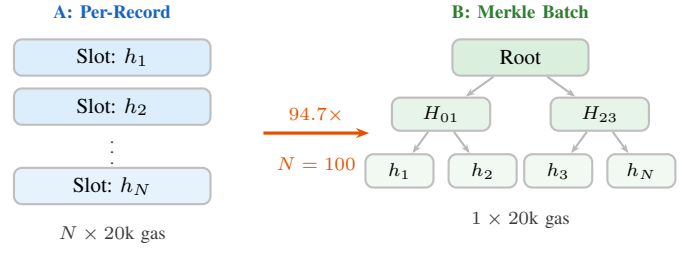


Fig. 1: Comparison of on-chain issuance models. Per-record storage writes one slot per diploma, whereas Merkle batching writes one root for the whole cohort.

TABLE I: Contract Variants

Label	Architecture	Key implementation detail
Packed	Per-record	Packed <code>uint256</code> slot: issuer address (160b) + revocation flag (1b)
Baseline	Merkle batch	Hand-written proof traversal and leaf construction using <code>abi.encodePacked</code>
OpenZeppelin	Merkle batch	Uses <code>OpenZeppelin MerkleProof.verifyCalldata</code>
Optimized	Merkle batch	Four EVM-level optimizations (T1–T4) applied to the verification hot path

T2 — Scratch-space keccak256. Instead of `abi.encodePacked(a,b)`, the two 32-byte inputs are written directly to EVM scratch space and hashed in place, avoiding repeated memory allocation.

T3 — Validation path splitting. The public `merkleLeaf()` performs input checks, while a private `_merkleLeaf()` is used internally.

T4 — Assembly preimage packing. The 112-byte leaf preimage is packed manually with `mstore/shl` instead of `abi.encodePacked`.

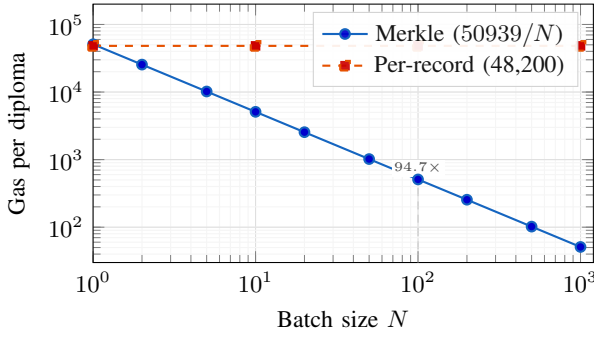
D. Experimental Setup

All measurements were collected with Foundry [7] using Solidity 0.8.20 with the optimizer enabled at 200 runs. Merkle trees were constructed on-chain only inside the test harness; in deployment they are built off-chain. Proof depths $d \in \{0, 1, 4, 8, 10, 12\}$ correspond to batch sizes from 2^0 to 2^{12} (1 to 4,096 diplomas). Gas was measured with `forge test -gas-report`; the reported values are function-level gas figures from Foundry’s gas report rather than full end-to-end test-harness cost.

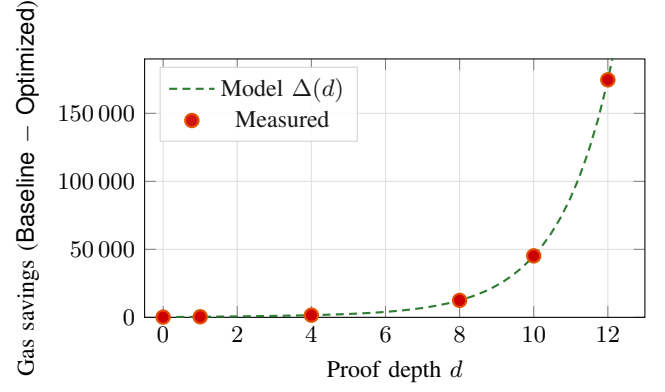
IV. RESULTS

A. Issuance Cost

Table II compares per-diploma issuance cost. In the Merkle architecture, publishing one batch root costs 50,939 gas regardless of batch size, so the amortized cost is $50,939/N$. In the per-record architecture, each diploma costs 48,200 gas.



(a) Per-diploma issuance cost. **Packed** is constant at 48,200 gas, while the Merkle cost decays as $50,939/N$.



(b) Closed-form model (3) versus measured gas savings (Baseline – Optimized).

TABLE II: Per-Diploma Issuance Cost

N	Merkle (gas)	Per-record (gas)	Ratio
1	50,939	48,200	$0.95\times$
10	5,094	48,200	$9.5\times$
50	1,019	48,200	$47\times$
100	509	48,200	$94.7\times$
1,000	51	48,200	$945\times$

TABLE III: Verification Gas (status statusWithProof)

Variant	Min	Avg	Max	vs. Packed
Packed	2,720 (constant)			—
Optimized	10,417	11,104	12,103	$4.1\times$
OpenZeppelin	10,560	11,532	12,936	$4.2\times$
Baseline	10,561	11,896	13,841	$4.4\times$

At $N = 1$, Merkle batching is slightly more expensive than a single direct write. From $N \geq 2$, amortization dominates:

$$\text{gas}_{\text{Merkle}}(N) = \frac{50,939}{N}, \quad \text{gas}_{\text{Packed}} = 48,200 \quad (1)$$

At $N = 100$, Merkle issuance lowers the per-diploma cost by 98.9%.

B. Verification Cost

Merkle batching shifts part of the cost from issuance to verification. Table III summarizes verification gas for depths $d \in \{0, 1, 4, 8\}$.

Per-record verification costs only 2,720 gas because it requires one storage read and minimal bit manipulation. This is the central trade-off: *per-record storage is expensive to write but cheap to read, whereas Merkle batching is cheap to write but more expensive to prove.*

Among the Merkle variants, **Optimized** has the lowest average verification gas, reducing cost by 6.7% relative to **Baseline** and by 3.7% relative to **OpenZeppelin**. In the current

TABLE IV: Deployment and State-Changing Costs

Variant	Deploy	Revoke (avg)	Issue
Packed	393,596	29,302	48,200
Optimized	1,018,273	58,303	50,939
OpenZeppelin	1,036,010	56,871	50,939
Baseline	1,142,020	59,610	51,012

UniVerify design, a `Valid` outcome also depends on the issuer still being associated with an active university, so verification captures both proof membership and current issuer trust status.

C. Deployment and Revocation

Packed deploys at roughly one-third of the gas of the Merkle variants because its logic is much simpler. However, deployment is paid once, whereas issuance cost is paid for every diploma or batch. For systems serving many graduation cycles, repeated issuance cost is therefore the more important quantity.

D. Economic Interpretation

Under an illustrative cost model with gas price $g = 40$ gwei and ETH price $p = \$2,500$, the dollar cost of an operation is:

$$\text{USD} = \text{gas} \cdot g \cdot p \cdot 10^{-9} \quad (2)$$

This yields approximately \$4.82 per diploma for **Packed** issuance (48,200 gas) and \$5.09 for a single Merkle batch root (50,939 gas). At cohort scale, the difference becomes dramatic: issuing 100 diplomas costs about \$482 with per-record storage versus about \$5.09 with Merkle batching, while issuing 1,000 diplomas costs about \$4,820 versus about \$5.09.

E. Closed-Form Savings Model

The gas savings between **Baseline** and **Optimized** can be decomposed into contributions from the four optimization techniques. Let d denote proof depth:

$$\Delta(d) = \underbrace{2^d \cdot 42}_{T_4} + \underbrace{96}_{T_3} + \underbrace{d \cdot 199}_{T_1+T_2} + 42 \quad (3)$$

TABLE V: Closed-Form Model Validation (T1–T4 Savings)

d	Leaves	Model Δ	Measured	Residual
0	1	180	180	0
1	2	421	421	0
4	16	1,606	1,606	0
8	256	12,484	12,484	0
10	1,024	45,136	45,218	82 (0.18%)
12	4,096	174,558	174,641	83 (0.05%)

The model is validated in Table V against measured gas differences. Both the model and the measurements use `revokeFromBatch`, which exercises the same proof-verification hot path as `statusWithProof` but provides cleaner gas accounting.

For $d \leq 8$, the prediction is exact. At $d \in \{10, 12\}$, the residual remains below 0.2%.

V. DISCUSSION AND CONCLUSION

A. Main Engineering Trade-off

The dominant design decision is architectural. **Packed** minimizes verification cost but scales linearly with the number of issued diplomas. Merkle batching does the opposite: it makes issuance almost constant at batch level, but shifts part of the work to the verifier. For diploma registries this is often favorable, because issuance is a paid institutional operation, whereas verification is a read-oriented action repeated over the credential lifetime.

B. Optimization Hierarchy

The measurements reveal three layers of optimization:

- 1) **Architecture:** switching from per-record storage to Merkle batching reduces issuance cost by up to 98.9% per diploma.
- 2) **Code-level tuning:** moving from **Baseline** to **Optimized** reduces verification gas by 6.6% and deployment gas by 10.8%.
- 3) **Library choice:** using **OpenZeppelin** instead of **Baseline** reduces verification gas by about 3.1%.

Micro-optimizations matter, but they cannot compensate for an inefficient underlying cost model. The largest gain comes from reducing state writes.

C. Practical Implications

For small-scale or infrequent issuance, per-record storage remains attractive because of its simple logic and constant-time lookup. For semester- or cohort-scale institutional issuance, Merkle batching is the more practical design.

For most production contracts, **OpenZeppelin's** `MerkleProof` remains a sensible default because it captures a substantial portion of the available optimization while preserving readability and auditability. The additional gains of **Optimized** are most relevant when proof verification is a critical path.

D. Limitations and Future Work

This study focuses on gas efficiency rather than privacy, governance, or interoperability. Proof generation is assumed to be off-chain. The economic examples are illustrative and depend on gas price and ETH/USD rate. The analysis also does not include Layer 2 fee markets, calldata compression, or blob-based data availability. In addition, the packed per-record contract is a simplified benchmark baseline, so the architectural comparison should be interpreted primarily as a cost study rather than as a comparison of two feature-identical production systems.

Future work includes evaluating the same architecture on Layer 2 networks, studying calldata-sensitive designs under EIP-4844, and benchmarking Verkle-based verification once production EVM support becomes available.

E. Conclusion

This paper compared two deployable architectures for on-chain diploma verification in the UniVerify system. The main conclusion is straightforward: *for blockchain-backed academic credentials, architecture dominates micro-optimization*. A per-record design offers the cheapest lookup, but Merkle batching changes issuance from a linear-cost operation into an amortized cohort-level commitment. At $N = 100$, issuance becomes $94.7\times$ cheaper per diploma, and for larger cohorts the difference becomes institutionally significant.

The optimized Merkle implementation further shows that low-level EVM tuning produces measurable and fairly predictable savings. Overall, the results suggest that practical blockchain credential systems should prioritize batching first and optimize the verification path only when scale justifies the added implementation complexity.

REFERENCES

- [1] A. Ezell and J. Bear, “Degree Mills: The Billion-Dollar Industry That Has Sold Over a Million Fake Diplomas,” Prometheus Books, 2012.
- [2] A. Grech and A. F. Camilleri, “Blockchain in Education,” *JRC Science for Policy Report*, European Commission, 2017.
- [3] A. Alketbi, Q. Nasir, and M. A. Talib, “Blockchain for Government Services — Use Cases, Security Benefits and Challenges,” in *Proc. 15th Learn. Technol. Conf.*, 2018.
- [4] D. Lizcano, J. A. Lara, B. White, and S. Aljawarneh, “Decentralised technology for educational credentials,” *J. Comput. High. Educ.*, vol. 32, pp. 9–49, 2020.
- [5] R. C. Merkle, “A Digital Signature Based on a Conventional Encryption Function,” in *Advances in Cryptology — CRYPTO ’87*, LNCS, vol. 293, pp. 369–378, 1988.
- [6] V. Buterin et al., “EIP-2929: Gas Cost Increases for State Access Opcodes,” *Ethereum Improvement Proposals*, 2020.
- [7] Paradigm, “Foundry: Ethereum Application Development Toolkit,” <https://github.com/foundry-rs/foundry>, 2023.
- [8] G. Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger,” *Ethereum Yellow Paper*, rev. 2024.
- [9] OpenZeppelin, “OpenZeppelin Contracts — MerkleProof,” <https://github.com/OpenZeppelin/openzeppelin-contracts>, 2026.
- [10] J. Kuszmaul, “Verkle Trees,” 2018; V. Buterin et al., “EIP-6800: Ethereum state using a unified Verkle tree,” 2023.